

Reinforcement Learning and Markov Decision Processes: Mathematical Notes

Let's consider a very simple Markov Decision Process (MDP) to illustrate the algorithms.

A. MDP Definition

States (S): We define three states:

- S_0 : Start state
- S_1 : Intermediate state
- S_2 : Terminal (Goal) state. Once S_2 is reached, the episode ends, and no further actions or rewards occur.

Actions (A): In states S_0 and S_1 , the agent can take two actions:

- $A = \{\text{Left (L)}, \text{Right (R)}\}$
- In S_2 , no actions are possible (it's a terminal state).

Transition Probabilities ($P(s'|s, a)$):

- From S_0 :
 - If action **Left**: Always transition to S_1 .
 - * $P(S_1|S_0, L) = 1.0$
 - If action **Right**: Always transition to S_2 .
 - * $P(S_2|S_0, R) = 1.0$
- From S_1 :
 - If action **Left**:
 - * $P(S_0|S_1, L) = 0.8$
 - * $P(S_2|S_1, L) = 0.2$
 - If action **Right**:
 - * $P(S_1|S_1, R) = 0.1$
 - * $P(S_2|S_1, R) = 0.9$

Reward Function ($R(s, a, s')$):

- $R(S_0, L, S_1) = -1$
- $R(S_0, R, S_2) = +10$
- $R(S_1, L, S_0) = -1$
- $R(S_1, L, S_2) = +5$
- $R(S_1, R, S_1) = -2$
- $R(S_1, R, S_2) = +8$
- All other rewards are 0. (Reward upon entering S_2 is considered part of the transition reward).

Discount Factor (γ):

- $\gamma = 0.9$

B. Problem 1: Policy Evaluation (Calculating $V^\pi(s)$ and $Q^\pi(s, a)$ for a Given Policy)

Goal: Given a specific policy π , calculate the state-value function $V^\pi(s)$ for all states $s \in S$, and the action-value function $Q^\pi(s, a)$ for all state-action pairs (s, a) .

Given Policy (π):

- In S_0 : Always choose **Left** ($\pi(S_0) = L$)
- In S_1 : Always choose **Right** ($\pi(S_1) = R$)

Equations: Bellman Expectation Equations The value functions for a policy π are defined by:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

$$Q^\pi(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

For a deterministic policy, $\pi(a|s)$ is 1 for the chosen action and 0 otherwise, simplifying the $V^\pi(s)$ equation. Since S_2 is a terminal state, $V^\pi(S_2) = 0$. We need to solve for $V^\pi(S_0)$ and $V^\pi(S_1)$.

Step 1: Set up the system of equations for $V^\pi(s)$

For S_0 : Since $\pi(S_0) = L$:

$$\begin{aligned} V^\pi(S_0) &= \sum_{s'} P(s'|S_0, L) [R(S_0, L, s') + \gamma V^\pi(s')] \\ &= P(S_1|S_0, L) [R(S_0, L, S_1) + \gamma V^\pi(S_1)] \\ &= 1.0 \cdot [-1 + 0.9 \cdot V^\pi(S_1)] \\ \mathbf{V^\pi(S_0) = -1 + 0.9V^\pi(S_1)} \quad (1) \end{aligned}$$

For S_1 : Since $\pi(S_1) = R$:

$$\begin{aligned} V^\pi(S_1) &= \sum_{s'} P(s'|S_1, R) [R(S_1, R, s') + \gamma V^\pi(s')] \\ &= P(S_1|S_1, R) [R(S_1, R, S_1) + \gamma V^\pi(S_1)] \\ &\quad + P(S_2|S_1, R) [R(S_1, R, S_2) + \gamma V^\pi(S_2)] \\ &= 0.1 \cdot [-2 + 0.9 \cdot V^\pi(S_1)] + 0.9 \cdot [8 + 0.9 \cdot V^\pi(S_2)] \end{aligned}$$

Since $V^\pi(S_2) = 0$:

$$\begin{aligned} V^\pi(S_1) &= 0.1 \cdot (-2 + 0.9V^\pi(S_1)) + 0.9 \cdot (8 + 0) \\ &= -0.2 + 0.09V^\pi(S_1) + 7.2 \\ &= 7.0 + 0.09V^\pi(S_1) \\ V^\pi(S_1) - 0.09V^\pi(S_1) &= 7.0 \\ 0.91V^\pi(S_1) &= 7.0 \\ \mathbf{V^\pi(S_1) = \frac{7.0}{0.91} \approx 7.6923} \quad (2) \end{aligned}$$

Step 2: Solve for $V^\pi(s)$ Substitute (2) into (1):

$$\begin{aligned} V^\pi(S_0) &= -1 + 0.9 \cdot (7.6923) \\ &= -1 + 6.92307 \\ \mathbf{V^\pi(S_0) \approx 5.9231} \end{aligned}$$

So, for the given policy π :

- $V^\pi(S_0) \approx 5.9231$
- $V^\pi(S_1) \approx 7.6923$

- $V^\pi(S_2) = 0$

Step 3: Calculate $Q^\pi(s, a)$ For S_0 :

$$\begin{aligned} Q^\pi(S_0, L) &= P(S_1|S_0, L)[R(S_0, L, S_1) + \gamma V^\pi(S_1)] \\ &= 1.0 \cdot [-1 + 0.9 \cdot 7.6923] = -1 + 6.92307 = \mathbf{5.9231} \\ Q^\pi(S_0, R) &= P(S_2|S_0, R)[R(S_0, R, S_2) + \gamma V^\pi(S_2)] \\ &= 1.0 \cdot [10 + 0.9 \cdot 0] = \mathbf{10.0} \end{aligned}$$

For S_1 :

$$\begin{aligned} Q^\pi(S_1, L) &= P(S_0|S_1, L)[R(S_1, L, S_0) + \gamma V^\pi(S_0)] \\ &\quad + P(S_2|S_1, L)[R(S_1, L, S_2) + \gamma V^\pi(S_2)] \\ &= 0.8 \cdot [-1 + 0.9 \cdot 5.9231] + 0.2 \cdot [5 + 0.9 \cdot 0] \\ &= 0.8 \cdot [-1 + 5.33079] + 0.2 \cdot 5 \\ &= 0.8 \cdot 4.33079 + 1 = 3.464632 + 1 = \mathbf{4.4646} \\ Q^\pi(S_1, R) &= P(S_1|S_1, R)[R(S_1, R, S_1) + \gamma V^\pi(S_1)] \\ &\quad + P(S_2|S_1, R)[R(S_1, R, S_2) + \gamma V^\pi(S_2)] \\ &= 0.1 \cdot [-2 + 0.9 \cdot 7.6923] + 0.9 \cdot [8 + 0.9 \cdot 0] \\ &= 0.1 \cdot [-2 + 6.92307] + 7.2 \\ &= 0.1 \cdot 4.92307 + 7.2 = 0.492307 + 7.2 = \mathbf{7.6923} \end{aligned}$$

C. Problem 2: Value Iteration (Finding the Optimal Policy)

Goal: Find the optimal state-value function $V^*(s)$ and derive the optimal policy π^* for the given MDP.

Equations: Bellman Optimality Equations

$$\begin{aligned} V^*(s) &= \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \\ Q^*(s, a) &= \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \end{aligned}$$

The optimal policy $\pi^*(s)$ chooses the action that maximizes $Q^*(s, a)$:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

We will use iterative updates, starting with $V_0(s) = 0$ for all non-terminal states. Note that $V^*(S_2) = 0$.

Iteration 0: $V_0(S_0) = 0, V_0(S_1) = 0$.

Iteration 1:

$$\begin{aligned} V_1(S_0) &= \max\{Q(S_0, L), Q(S_0, R)\} \\ Q(S_0, L) &= P(S_1|S_0, L)[R(S_0, L, S_1) + \gamma V_0(S_1)] = 1.0 \cdot [-1 + 0.9 \cdot 0] = -1 \\ Q(S_0, R) &= P(S_2|S_0, R)[R(S_0, R, S_2) + \gamma V_0(S_2)] = 1.0 \cdot [10 + 0.9 \cdot 0] = 10 \\ V_1(S_0) &= \max(-1, 10) = 10 \\ V_1(S_1) &= \max\{Q(S_1, L), Q(S_1, R)\} \\ Q(S_1, L) &= P(S_0|S_1, L)[R(S_1, L, S_0) + \gamma V_0(S_0)] + P(S_2|S_1, L)[R(S_1, L, S_2) + \gamma V_0(S_2)] \\ &= 0.8 \cdot [-1 + 0.9 \cdot 0] + 0.2 \cdot [5 + 0.9 \cdot 0] = -0.8 + 1.0 = 0.2 \\ Q(S_1, R) &= P(S_1|S_1, R)[R(S_1, R, S_1) + \gamma V_0(S_1)] + P(S_2|S_1, R)[R(S_1, R, S_2) + \gamma V_0(S_2)] \\ &= 0.1 \cdot [-2 + 0.9 \cdot 0] + 0.9 \cdot [8 + 0.9 \cdot 0] = -0.2 + 7.2 = 7.0 \\ V_1(S_1) &= \max(0.2, 7.0) = 7.0 \end{aligned}$$

Summary after Iteration 1: $V_1(S_0) = 10, V_1(S_1) = 7.0$.

Iteration 2:

$$\begin{aligned}
 V_2(S_0) &= \max\{Q(S_0, L), Q(S_0, R)\} \\
 Q(S_0, L) &= 1.0 \cdot [-1 + 0.9 \cdot V_1(S_1)] = -1 + 0.9 \cdot 7.0 = -1 + 6.3 = 5.3 \\
 Q(S_0, R) &= 1.0 \cdot [10 + 0.9 \cdot V_1(S_2)] = 1.0 \cdot [10 + 0.9 \cdot 0] = 10.0 \\
 V_2(S_0) &= \max(5.3, 10.0) = 10.0 \\
 \\
 V_2(S_1) &= \max\{Q(S_1, L), Q(S_1, R)\} \\
 Q(S_1, L) &= 0.8 \cdot [-1 + 0.9 \cdot V_1(S_0)] + 0.2 \cdot [5 + 0.9 \cdot V_1(S_2)] \\
 &= 0.8 \cdot [-1 + 0.9 \cdot 10.0] + 0.2 \cdot [5 + 0] \\
 &= 0.8 \cdot 8.0 + 1.0 = 6.4 + 1.0 = 7.4 \\
 Q(S_1, R) &= 0.1 \cdot [-2 + 0.9 \cdot V_1(S_1)] + 0.9 \cdot [8 + 0.9 \cdot V_1(S_2)] \\
 &= 0.1 \cdot [-2 + 0.9 \cdot 7.0] + 0.9 \cdot [8 + 0] \\
 &= 0.1 \cdot [-2 + 6.3] + 7.2 = 0.1 \cdot 4.3 + 7.2 = 0.43 + 7.2 = 7.63 \\
 V_2(S_1) &= \max(7.4, 7.63) = 7.63
 \end{aligned}$$

Summary after Iteration 2: $V_2(S_0) = 10.0$, $V_2(S_1) = 7.63$.

Iteration 3:

$$\begin{aligned}
 V_3(S_0) &= \max\{Q(S_0, L), Q(S_0, R)\} \\
 Q(S_0, L) &= 1.0 \cdot [-1 + 0.9 \cdot V_2(S_1)] = -1 + 0.9 \cdot 7.63 = -1 + 6.867 = 5.867 \\
 Q(S_0, R) &= 10.0 \quad (\text{no change as } V(S_2) = 0) \\
 V_3(S_0) &= \max(5.867, 10.0) = 10.0 \\
 \\
 V_3(S_1) &= \max\{Q(S_1, L), Q(S_1, R)\} \\
 Q(S_1, L) &= 0.8 \cdot [-1 + 0.9 \cdot V_2(S_0)] + 0.2 \cdot [5 + 0.9 \cdot V_2(S_2)] \\
 &= 0.8 \cdot [-1 + 0.9 \cdot 10.0] + 0.2 \cdot [5 + 0] = 0.8 \cdot 8.0 + 1.0 = 7.4 \\
 Q(S_1, R) &= 0.1 \cdot [-2 + 0.9 \cdot V_2(S_1)] + 0.9 \cdot [8 + 0.9 \cdot V_2(S_2)] \\
 &= 0.1 \cdot [-2 + 0.9 \cdot 7.63] + 0.9 \cdot [8 + 0] \\
 &= 0.1 \cdot [-2 + 6.867] + 7.2 = 0.1 \cdot 4.867 + 7.2 = 0.4867 + 7.2 = 7.6867 \\
 V_3(S_1) &= \max(7.4, 7.6867) = 7.6867
 \end{aligned}$$

Summary after Iteration 3: $V_3(S_0) = 10.0$, $V_3(S_1) = 7.6867$.

Values are converging. $V_2(S_0) = 10.0 \rightarrow V_3(S_0) = 10.0$ (converged for S_0). $V_2(S_1) = 7.63 \rightarrow V_3(S_1) = 7.6867$ (still converging, but very close to the optimal $V^*(S_1) \approx 7.6923$). For practical purposes, we often stop when the maximum change in value functions across all states falls below a small threshold (e.g., 10^{-3}).

Deriving the Optimal Policy (π^*): Once $V^*(s)$ has converged, the optimal policy $\pi^*(s)$ for each state s is the action a that yields the maximum $Q^*(s, a)$, where $Q^*(s, a)$ is calculated using the converged $V^*(s')$:

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

Using the converged $V^*(S_0) = 10.0$ and $V^*(S_1) \approx 7.6923$:

For S_0 :

$$\begin{aligned}
 Q^*(S_0, L) &= -1 + 0.9 \cdot V^*(S_1) = -1 + 0.9 \cdot 7.6923 \approx 5.9231 \\
 Q^*(S_0, R) &= 10 + 0.9 \cdot V^*(S_2) = 10 + 0.9 \cdot 0 = 10.0
 \end{aligned}$$

Since $10.0 > 5.9231$, $\pi^*(S_0) = R$.

For S_1 :

$$\begin{aligned}
 Q^*(S_1, L) &= 0.8 \cdot [-1 + 0.9 \cdot V^*(S_0)] + 0.2 \cdot [5 + 0.9 \cdot V^*(S_2)] \\
 &= 0.8 \cdot [-1 + 0.9 \cdot 10.0] + 0.2 \cdot [5 + 0] = 0.8 \cdot 8.0 + 1.0 = 7.4 \\
 Q^*(S_1, R) &= 0.1 \cdot [-2 + 0.9 \cdot V^*(S_1)] + 0.9 \cdot [8 + 0.9 \cdot V^*(S_2)] \\
 &= 0.1 \cdot [-2 + 0.9 \cdot 7.6923] + 0.9 \cdot [8 + 0] \\
 &\approx 0.1 \cdot [-2 + 6.92307] + 7.2 = 0.1 \cdot 4.92307 + 7.2 \approx 7.6923
 \end{aligned}$$

Since $7.6923 > 7.4$, $\pi^*(S_1) = R$.

Optimal Policy π^* :

- $\pi^*(S_0) = \text{Right}$
- $\pi^*(S_1) = \text{Right}$

This suggests the optimal strategy is to always try to go ‘Right’ to reach the terminal state S_2 as quickly as possible, even if it sometimes means staying in S_1 (with action R in S_1 , there’s a 0.1 chance of staying in S_1 and a 0.9 chance of going to S_2).

D. Problem 3: Q-Learning (Model-Free Learning)

Goal: Understand how an agent learns Q-values through interaction with the environment, without explicit knowledge of transition probabilities or rewards (model-free).

Equation: Q-Learning Update Rule The Q-value for a state-action pair (s, a) is updated after experiencing a transition (s, a, r, s') :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- α : Learning rate (how much to update Q-value based on new experience). Let’s use $\alpha = 0.1$.
- r : Immediate reward received.
- γ : Discount factor (same as before, $\gamma = 0.9$).
- s' : Next state.
- $\max_{a'} Q(s', a')$: The maximum Q-value in the next state s' . If s' is a terminal state, this term is 0.

Initial Q-Table: We start with all Q-values initialized to 0.

State	Q(s, Left)	Q(s, Right)
S_0	0	0
S_1	0	0

Simulated Experience (Agent interacts with the environment):

Let’s trace the Q-value updates for a few interaction steps. Assume the agent uses an ϵ -greedy policy for exploration (e.g., usually picks the best known action, but occasionally picks a random one). For this example, we’ll just *show* the experiences.

Experience 1: $(S_0, R, +10, S_2)$

- Agent is in S_0 , takes action **Right**, receives reward +10, transitions to S_2 (terminal).
- Update $Q(S_0, R)$:

$$Q(S_0, R) \leftarrow Q(S_0, R) + \alpha \left[r + \gamma \max_{a'} Q(S_2, a') - Q(S_0, R) \right]$$

Since S_2 is terminal, $\max_{a'} Q(S_2, a') = 0$.

$$\begin{aligned}
 Q(S_0, R) &\leftarrow 0 + 0.1 [10 + 0.9 \cdot 0 - 0] \\
 Q(S_0, R) &\leftarrow 0.1 \cdot 10 = \mathbf{1.0}
 \end{aligned}$$

Q-Table after Experience 1:

State	Q(s, Left)	Q(s, Right)
S_0	0	1.0
S_1	0	0

Experience 2: ($S_0, \mathbf{L}, -1, S_1$)

- Agent is in S_0 , takes action **Left**, receives reward -1 , transitions to S_1 .
- Update $Q(S_0, \mathbf{L})$:

$$Q(S_0, \mathbf{L}) \leftarrow Q(S_0, \mathbf{L}) + \alpha \left[r + \gamma \max_{a'} Q(S_1, a') - Q(S_0, \mathbf{L}) \right]$$

Here, $\max_{a'} Q(S_1, a') = \max(Q(S_1, \mathbf{L}), Q(S_1, \mathbf{R})) = \max(0, 0) = 0$.

$$Q(S_0, \mathbf{L}) \leftarrow 0 + 0.1 [-1 + 0.9 \cdot 0 - 0]$$

$$Q(S_0, \mathbf{L}) \leftarrow 0.1 \cdot (-1) = -\mathbf{0.1}$$

Q-Table after Experience 2:

State	Q(s, Left)	Q(s, Right)
S_0	-0.1	1.0
S_1	0	0

Experience 3: ($S_1, \mathbf{R}, -2, S_1$)

- Agent is in S_1 , takes action **Right**, receives reward -2 , transitions to S_1 .
- Update $Q(S_1, \mathbf{R})$:

$$Q(S_1, \mathbf{R}) \leftarrow Q(S_1, \mathbf{R}) + \alpha \left[r + \gamma \max_{a'} Q(S_1, a') - Q(S_1, \mathbf{R}) \right]$$

Here, $\max_{a'} Q(S_1, a') = \max(Q(S_1, \mathbf{L}), Q(S_1, \mathbf{R})) = \max(0, -0.2) = 0$.

$$Q(S_1, \mathbf{R}) \leftarrow 0 + 0.1 [-2 + 0.9 \cdot 0 - 0]$$

$$Q(S_1, \mathbf{R}) \leftarrow 0.1 \cdot (-2) = -\mathbf{0.2}$$

Q-Table after Experience 3:

State	Q(s, Left)	Q(s, Right)
S_0	-0.1	1.0
S_1	0	-0.2

Experience 4: $(S_1, \mathbf{R}, +8, S_2)$

- Agent is in S_1 , takes action **Right**, receives reward +8, transitions to S_2 (terminal).
- Update $Q(S_1, \mathbf{R})$:

$$Q(S_1, \mathbf{R}) \leftarrow Q(S_1, \mathbf{R}) + \alpha \left[r + \gamma \max_{a'} Q(S_2, a') - Q(S_1, \mathbf{R}) \right]$$

Since S_2 is terminal, $\max_{a'} Q(S_2, a') = 0$.

$$Q(S_1, \mathbf{R}) \leftarrow -0.2 + 0.1 [8 + 0.9 \cdot 0 - (-0.2)]$$

$$Q(S_1, \mathbf{R}) \leftarrow -0.2 + 0.1 [8 + 0.2]$$

$$Q(S_1, \mathbf{R}) \leftarrow -0.2 + 0.1 \cdot 8.2 = -0.2 + 0.82 = \mathbf{0.62}$$

Q-Table after Experience 4:

State	Q(s, Left)	Q(s, Right)
S_0	-0.1	1.0
S_1	0	0.62

Experience 5: $(S_0, \mathbf{L}, -1, S_1)$ (Let's re-visit S_0 after S_1 has seen some updates)

- Agent is in S_0 , takes action **Left**, receives reward -1, transitions to S_1 .
- Update $Q(S_0, \mathbf{L})$:

$$Q(S_0, \mathbf{L}) \leftarrow Q(S_0, \mathbf{L}) + \alpha \left[r + \gamma \max_{a'} Q(S_1, a') - Q(S_0, \mathbf{L}) \right]$$

Here, $\max_{a'} Q(S_1, a') = \max(Q(S_1, \mathbf{L}), Q(S_1, \mathbf{R})) = \max(0, 0.62) = 0.62$.

$$Q(S_0, \mathbf{L}) \leftarrow -0.1 + 0.1 [-1 + 0.9 \cdot 0.62 - (-0.1)]$$

$$Q(S_0, \mathbf{L}) \leftarrow -0.1 + 0.1 [-1 + 0.558 + 0.1]$$

$$Q(S_0, \mathbf{L}) \leftarrow -0.1 + 0.1 [-0.342]$$

$$Q(S_0, \mathbf{L}) \leftarrow -0.1 - 0.0342 = \mathbf{-0.1342}$$

Q-Table after Experience 5:

State	Q(s, Left)	Q(s, Right)
S_0	-0.1342	1.0
S_1	0	0.62

Discussion:

- **Convergence:** When running RL algorithms like Q-Learning, **Q-values** gradually update over time. Under certain conditions (like sufficient exploration and a decaying learning rate), Q-Learning is guaranteed to converge to the optimal Q-values, $Q^*(s, a)$.
- **Off-Policy Learning:** Q-Learning is an **off-policy algorithm**. This means its update rule for $Q(s, a)$ uses $\max_{a'} Q(s', a')$, considering the *best possible action* in the next state (s'). It doesn't matter what specific action the agent actually *took* to reach s' or what exploration policy it's currently following. This makes Q-Learning powerful because it can learn the optimal policy while still exploring different paths.
- **Comparison to Value Iteration:** While **Value Iteration** directly calculates optimal state values $V^*(s)$ by iterating through the Bellman Optimality Equation (requiring full knowledge of transition probabilities $P(s'|s, a)$ and rewards $R(s, a, s')$), **Q-Learning** is **model-free**. It learns optimal action-values ($Q^*(s, a)$) solely from observed experiences (state, action, reward, next state tuples) without needing a pre-built model of the environment.

- **Role of α and γ :**
 - The **learning rate** (α) controls how much new information overrides old information during updates. A higher α means faster but potentially less stable learning.
 - The **discount factor** (γ) determines the importance of future rewards. A γ closer to 1 emphasizes long-term rewards, while a smaller γ prioritizes immediate rewards.
- **Exploration-Exploitation Trade-off:** In any real-world RL agent, balancing **exploration** (trying new actions to discover better rewards) and **exploitation** (choosing the best-known action based on current knowledge) is crucial. Strategies like **ϵ -greedy** are vital; without sufficient exploration, the agent might get stuck in suboptimal policies because it never tries actions that could lead to higher, undiscovered rewards.

Prepared By:

Md. Atikuzzaman

Lecturer

Department of Computer Science and Engineering

Green University of Bangladesh

Email: atik@cse.green.edu.bd